

Code review — mostenire si polimorfism

Runde de review pe proiectul tau, cea mai noua sus.

Runda 2 — 2026-09-23 — sha 335c45e

Ai livrat doua teme intregi: **desenele geometrice** (pachetul `formeGeometrice`) si **orarul unei zile** (pachetul `OrarScheduler`), plus ai inchis B1 din runda 1.

Poarta de compilare: **trece**. `javac` pe toate sursele iese cu 0 erori si `java app.App` porneste. E prima livrare a ta care compileaza din prima — `getRol()` in loc de `u.rol` a rezolvat exact ce trebuia.

Ce e bine, si nu e putin: ierarhiile sunt proiectate corect (superclasa tine ce e comun, subclasele adauga), ai scris constructor de copiere pe fiecare nivel cu `super(copie)`, iar `getTipInterval()` / `afisare()` sunt suprascrise si legate la rulare, nu la compilare. Ai inteles mecanismul.

Problema rundeii e alta, si e una singura, repetata in doua pachete: **metodele de copiere pe care le-ai scris nu sunt chemate de nimeni**, iar ce trece drept „duplicare” imparte obiectele cu originalul — sau, la orar, chiar il goleste. Tot ce scriu mai jos e reproduc prin rulare, nu dedus.

Critice

B1  `duplicate()` goleste orarul din care copiezi

`src/app/OrarScheduler/Orar.java:15–17` · acelasi bug la `Scheduler.java:31–33`

CUM E ACUM: un orar cu 2 intervale, dupa un singur `duplicate()` :

```
sursa inainte, nr intervale = 2
sursa dupa duplicate, nr intervale = 0
copie, nr intervale = 0
afisarea sursei dupa duplicare:
nu exista niciun interval setat.
```

Orarul original a ramas gol, iar copia s-a nascut goala. Datele nu s-au copiat nicaieri: au disparut.

DE CE: constructorul `Orar(Orar orar)` ruleaza **pe obiectul nou** — `this` e copia, iar campul `arrIntervale` din interiorul lui e lista goala tocmai initializata la linia 9. Tu scrii `orar.setArrIntervale(arrIntervale)`, adica „ia lista **mea**, cea goala, si pune-o in **sursa**”. Directia e inversata: sensul corect e dinspre sursa spre `this`. Dupa apel, ambele referinte arata catre aceeasi lista goala, deci pierzi si originalul.

Dar chiar si scris in directia buna (`this.arrIntervale = orar.arrIntervale`) ai avea o copie falsa: doua obiecte `Orar` care se uita la **aceeasi** lista. Adaugi un interval in copie, apare si in original. O copie adevarata cere trei lucruri: o lista **noua**, parcurgerea sursei, si cate un obiect **nou** pentru fiecare interval — adica exact `interval.duplicate()`, metoda pe care ai scris-o pe toate cele trei subclase si pe care **nu o cheama nimeni** in tot proiectul. Acolo isi merita polimorfismul banii: tu scrii un singur `duplicate()` in bucla, iar fiecare element stie singur ce tip de copie sa produca.

FIX:

```
public Orar(Orar orar) {
    for (Interval interval : orar.arrIntervale) {
        this.arrIntervale.add(interval.duplicate());
    }
}
```

`Scheduler.duplicare(orar)` are acelasi efect asupra orarului primit — l-am rulat, orarul trimis ramane cu 0 intervale.

B2 Decalarea nu schimba nimic: minutele se pierd la salvare

`src/app/OrarScheduler/Interval.java:26-28` si `:64-88`

CUM E ACUM: rulez decalarea si citesc starea inapoi:

```
inainte, getOraInceput() = 10
Orarul a fost decalat. Noua ora de incepere CURS este ora 10:05
dupa decalare(5), getOraInceput() = 10
dupa decalare(45), getOraInceput() = 10
```

Mesajul spune 10:05, obiectul a ramas la 10:00. Se vede si in rularea App-ului tau: decalezi cu 5 minute, apoi `afisare()` tipareste tot ora fixa.

DE CE: campul `oraInceput` tine **minute**, dar cele doua metode care il ating se contrazic. `setOraInceput` inmulteste cu 60 („primesc ore”), iar `getOraInceput` imparte la 60 si, fiind impartire intreaga intre `int`-uri, taie restul. Rezultatul: prin getter nu poate iesi niciodata altceva decat o ora fixa.

În `decalare` calculezi corect: $10 \cdot 60 + 5 = 605$ minute. Apoi faci $605 / 60 = 10$ și chemi `setOraInceput(10)`, care înmulțește înapoi cu 60 și scrie 600. Minutele sunt aruncate exact în pasul de salvare — nu în calcul.

Regula generală: **unitatea de măsură se alege o singură dată**, la câmp. Dacă `oraInceput` e în minute de la miezul nopții, setterul primește minute și getterul da minute; împărțirea la 60 apare o singură dată, în clipa în care scrii pe ecran.

FIX: ține minutele așa cum vin (`this.oraInceput = minute;`), da un constructor care primește (`ora, minut`) și face `ora * 60 + minut`, iar `decalare` devine o linie: `oraInceput = oraInceput + nrMinute;`.

B3  `afisare()` tipărește ora de **sfârșit**, nu cea de început

`src/app/OrarScheduler/Interval.java:43-62`

CUM E ACUM: trei intervale, așa cum le construiești tu în `App`:

```
CURS: ora 11:00 Engleza 124B      <- cursul incepe la 10, dureaza 60 min
PAUZA: ora 13:00
ACTIVITATE: ora 30:40 Sport TerenFotbal
```

Cursul de la 10:00 apare ca „ora 11:00”, iar activitatea de la 14:00 cu 1000 de minute apare la „ora 30:40” — o ora care nu există într-o zi.

DE CE: în `afisare()` faci `ora += getDurataMin() / 60` și `minute = getDurataMin() % 60`. Asta nu formatează ora de început: adună durata peste ea, deci calculează **finalul** intervalului. Ora de început, singura pe care cerința ți-o cere, nu se tipărește nicaieri. Iar când suma depășește 24 de ore nimeni nu o aduce înapoi în intervalul unei zile, de unde „30:40”.

Amanuntul care leagă B2 de B3: câmpul și afisarea folosesc două convenții diferite (`60` înseamnă „o ora” într-un loc și „durată” în altul). Când ai aceeași valoare interpretată în două feluri, greselile se ascund una pe alta — decalarea pare că merge pentru că afisarea oricum arată altceva.

FIX: formatează separat începutul și sfârșitul, amândouă din minute: `inceput = oraInceput`, `sfarsit = oraInceput + durataMin`, fiecare trecut prin aceeași metodă mică de formatare (`h = m / 60`, `min = m % 60`, cu zero în față când `min < 10`).

B4  Un orar cu un singur interval raportează că e gol

`src/app/OrarScheduler/Orar.java:39`

CUM E ACUM:

La `Desen` e si mai vizibil: adaug o figura in lista originala si ea apare si in copie, desi copia a fost facuta inainte.

DE CE: `new Linie(punctA, punctB)` copiaza **referintele**, nu punctele. Dupa apel exista doua obiecte `Linie`, dar o singura pereche de obiecte `Punct`, la care se uita amandoua. Orice modificare prin oricare drum se vede prin celalalt. La fel `new Desen(this.figuri)`: o singura lista, doua desene.

Cheia e ca tu **ai deja** rezolvarea scrisa: `Punct.duplicare()` chiar construiesc un `Punct` nou. Doar ca nimeni nu o cheama — la fel ca `Interval.duplicate()` din B1. Tiparul se repeta in doua pachete diferite, deci nu e o scapare punctuala, e regula care trebuie fixata: **cand copiezi un obiect care contine alte obiecte, copiezi si continutul**, iar continutul si-l copiezi singur, polimorfic.

FIX: `new Linie((Punct) punctA.duplicare(), (Punct) punctB.duplicare());` si, la `Desen`, lista noua plus `fig.duplicare()` pentru fiecare figura.

B7  **Al doilea constructor al `Eticheta` lasa obiectul pe jumatate construit**

`src/app/formeGeometrice/Eticheta.java:36-42`

CUM E ACUM:

```
new Eticheta(new Punct(0,0), "text", new Punct(5,5)).afisare();
-> java.lang.NullPointerException: Cannot invoke
    "app.formeGeometrice.Dreptunghi.toString()" because "this.dreptunghi" is null
```

DE CE: constructorul cu trei parametri cheama `super(punct1, punct2)` si pune textul, dar nu atinge campul `dreptunghi` — linia care l-ar fi construit e comentata (:40). Java nu are cum sa te avertizeze: un camp de tip obiect neinitializat e `null`, nu o eroare de compilare. Se afla abia cand cineva il foloseste, iar primul care il foloseste e `toString()`.

Cauza adanca nu e linia comentata, ci ca `Eticheta` tine aceeasi informatie in doua locuri (vezi M3): o data mostenita din `Dreptunghi` si o data in campul `dreptunghi`. Cu doua copii ale aceluiasi lucru, orice constructor nou are doua sanse sa uite una.

FIX: dupa ce aplici M3 (scoti campul), constructorul devine `super(p1, p2) + setText(text)` si nu mai are ce sa uite.

B8  **`Punct.equals` crapa pe orice alt tip si pe `null`**

`src/app/formeGeometrice/Punct.java:46-49`

CUM E ACUM:

```
equals(String) -> java.lang.ClassCastException: class java.lang.String
    cannot be cast to class app.formeGeometrice.Punct
equals(null) -> java.lang.NullPointerException: Cannot read field "x"
```

DE CE: semnatura e `equals(Object o)` — deci metoda ta promite ca stie sa raspunda pentru **orice** obiect, inclusiv `null`. Tu faci cast direct la `Punct`, fara sa verifici. Contractul lui `equals` cere trei lucruri inainte de comparare: `null` inseamna `false`, alt tip inseamna `false`, si abia apoi compari campurile.

De ce conteaza in practica: `List.contains`, `indexOf` si `remove` cheama `equals` pe elemente. `lista.contains("text")` mi-a raspuns `false` fara sa crape doar din noroc — acolo se cheama `"text".equals(punct)`, deci `equals` -ul lui `String`, nu al tau. Daca inversezi ordinea (`punct.equals("text")`) sau daca lista contine si altceva, primesti exceptie in mijlocul unei operatii banale de cautare.

FIX:

```
@Override
public boolean equals(Object o) {
    if (!(o instanceof Punct)) {
        return false;
    }
    Punct alt = (Punct) o;
    return x == alt.getX() && y == alt.getY();
}
```

Importante

M1 🟡 Mesajele decalarii: text de test livrat, minute inversate, mesaj dublu

`src/app/OrarScheduler/Interval.java:76-86`

Trei lucruri, toate reproduse:

```
decalare(60) de la ora 8 -> doua mesaje:
    ... este ora 9:0
    ... este ora 9:00
decalare(5) de la ora 8 -> ... este ora 8:50test
```

`0test` a ramas din depanare si a ajuns in commit. „8:50” ar trebui sa fie „8:05” — zeroul se lipeste dupa minute intr-o ramura si inainte in alta. Iar mesajul dublu vine din `if / if / else` : primul `if` e independent, al doilea formeaza pereche cu `else` , deci cand decalajul cade fix pe ora intreaga intra si primul `if` , si `else` -ul. Cu `if / else if / else` ai fi avut exact o ramura.

Separat de asta: `decalare afiseaza`. O metoda care modifica starea nu ar trebui sa si tipareasca — cine o cheama in bucla (`decalare0rar`) nu mai poate alege formatul, si nici nu poate decala in tacere. Lasa afisarea pe seama lui `afisare()` .

M2 🟡 `Scheduler` e o clasa moarta

`src/app/0rarScheduler/Scheduler.java:8-33`

`lista0rare` nu e initializat niciodata: `getLista0rare()` intoarce `null` , iar `setLista0rare()` crapa cu `NullPointerException` (reprodus). `creeaza0rar()` si `duplicare()` construiesc obiecte in variabile locale si le arunca la iesirea din metoda — nimic nu se salveaza nicaieri.

E al treilea rand in care apare acelasi tipar la tine: cod scris corect ca sintaxa, dar pe care nu-l cheama si nu-l verifica nimeni. Regula de lucru: **dupa ce scrii o metoda, cheam-o macar o data din `App` si uita-te la ce iese**. Daca nu-ti trebuie inca, nu o scrie — o adaugi cand o folosesti.

M3 🟡 `Eticheta` tine acelasi dreptunghi in doua locuri

`src/app/formeGeometrice/Eticheta.java:3-7`

Clasa **extinde** `Dreptunghi` (deci mosteneste `punctA / punctB`) si, in plus, are un camp `dreptunghi` . Sunt doua stari care trebuie tinute in sincron cu mana:

```
mostenit : X:0,Y:0      camp: X:0,Y:0
dupa setDreptunghi(...)
mostenit : X:0,Y:0      camp: X:99,Y:99
dupa translate(1,1)
mostenit : X:0,Y:0      camp: X:1,Y:1
```

`translate` muta doar campul, `toString` citeste doar campul, iar partea mostenita ramane in urma pentru totdeauna. Alege una: **ori** `Eticheta extends Dreptunghi` si folosesti `punctA / punctB` mostenite (atunci scoti campul), **ori** `Eticheta are un Dreptunghi` si nu-l mai extinde. Intrebarea care decide: „o eticheta **este** un dreptunghi?” Daca da, mostenire; daca doar il foloseste ca sa se deseneze, compozitie.

M4 🟡 Toata afisarea trece prin `toString()` , desi fisa il excludea

`Punct.java:39` · `Linie.java:35` · `Dreptunghi.java:34` · `Cerc.java:36` · `Eticheta.java:45` · `Desen.java:16`

`afisare()` a ajuns un invelis peste `toString()` (`System.out.println(this)`). Nu e gresit ca program, dar rateaza exercitiul: tema cerea afisarea prin metoda ta, suprascrisa pe fiecare subclasa, tocmai ca sa vezi legarea tarzie fara ajutorul mecanismului automat din Java. In plus `toString()` e chemat implicit de `println`, de depanator si de concatenari — daca il folosesti si ca format de afisare, nu mai ai unde sa-l schimbi cand ai nevoie de doua formate diferite.

La orar ai facut-o corect: `afisare()` intoarce `String` si `Orar` decide unde il scrie. Adu si desenele la forma asta.

M5 🟡 `Orar` isi expune lista si nu-si apara continutul

`src/app/OrarScheduler/Orar.java:9` si `:20-22`

Campul `arrIntervale` nu e `private` (e vizibil in tot pachetul), iar `setArrIntervale` e `public` — adica oricine poate inlocui lista dintr-o bucata. Exact prin usa asta a intrat B1. `getArrIntervale()` e pe drumul bun (intoarce o lista noua), dar copiaza doar **lista**, nu si intervalele: cine primeste rezultatul poate modifica intervalele orarului tau prin referintele copiate.

Cleanups

- **C1** `App.java:91` : `System.out.println("test");` ramas in cod.
 - **C2** `Interval.java:67-69` : `if` cu corp gol (verificarea peste 1440 de minute nu e scrisa) — ori o completezi, ori o stergi.
 - **C3** `OraCurs.java:23-26` : varianta veche a lui `afisare()` ramasa comentata. Istoricul il tine git, nu fisierul.
 - **C4** `ActivitateLibera.java:7-8` : `denumire` si `locDesfasurare` fara `private`.
 - **C5** pachetul `app.OrarScheduler` : conventia Java cere pachete cu litere mici (`app.orar`). Numele claselor raman cu majuscula.
 - **C6** `Cerc` nu e folosit nicaieri in `App` — figura exista, dar n-ai vazut-o niciodata rulata.
 - **C7** `Desen.toString():18-20` : concatenare cu `+=` in bucla — pentru desene mari se aduna obiecte `String` inutile; `StringBuilder` e varianta obisnuita.
 - **C8** `Pauza.afisare():24-26` cheama doar `super.afisare()` — `@Override` -ul nu adauga nimic si poate fi sters.
 - **C9** `Figura.duplicare():15` intoarce `null` si are `;` in plus dupa acolada. Cat timp nu folosim `abstract`, pune-i macar un comportament sigur.
-

Restante din runda 1

-  **B1 inchis** — `getRol()` in loc de `u.rol`, proiectul compileaza.
-  **B4 pe jumătate** — `numarStudent()` exista acum (`UserRepository`), dar exercitiul 8 (`ClassCastException` provocata intentionat) tot nu e scris.
-  **B2 NEATINS** — `loadData()` n-are `case "ADMIN"` (`UserRepository.java:46-53`). Un `Admin` salvat dispare la urmatoarea pornire, iar `save()` rescrie fisierul fara el.
-  **B3 NEATINS** — `try` e tot in jurul intregului `while` (`UserRepository.java:41-57`): prima linie stricata opreste citirea restului.
-  M1-M4 din runda 1 neatinse (exercitiile 2 si 3 amestecate, `nume` duplicat in subclase, `cautaUserDupaId` care si cauta si tipareste, `Scanner` — acesta din urma **inchis**, e in `try (...)`).

Before / after (critice)

#	Cum e acum	Cum ar trebui
B1	<code>orar.setArrIntervale(arrIntervale);</code>	bucla peste sursa cu <code>this.arrIntervale.add(interval.duplicate());</code>
B2	<code>setOraInceput(oraDecalata)</code> dupa <code>/ 60</code>	<code>oraInceput = oraInceput + nrMinute;</code> , campul in minute
B3	<code>ora += getDurataMin() / 60</code> in <code>afisare()</code>	<code>inceput = oraInceput</code> , <code>sfarsit = oraInceput + durataMin</code>
B4	<code>if (arrIntervale.size() > 1)</code>	<code>if (arrIntervale.isEmpty()) { mesaj; return; }</code>
B5	<code>this.setX(x); this.setY(y);</code>	<code>this.x += x; this.y += y;</code>
B6	<code>new Linie(punctA, punctB)</code>	<code>new Linie((Punct) punctA.duplicare(), (Punct) punctB.duplicare())</code>
B7	camp <code>dreptunghi</code> neinitializat in constructorul cu 3 parametri	scoate campul (M3); ramane <code>super(p1, p2) + setText(text)</code>
B8	<code>Punct p = (Punct) o;</code> direct	<code>if (!(o instanceof Punct)) return false;</code> inainte de cast

Q&A — raspunde in scris, doua-trei randuri fiecare

1. **B1 + B6.** Ai scris `duplicate()` in `Interval` si `duplicare()` in `Punct` , amandoua corecte, si in niciunul din pachete nu le cheama nimeni. Ce ar trebui sa faci, ca obicei de lucru, ca sa prinzi singur genul asta de cod — scris, dar nefolosit?
2. **B2.** Campul `oraInceput` tine minute, dar `getOraInceput()` intoarce ore. Daca pastrezi ambele metode exact asa cum sunt, exista vreo valoare de decalaj pentru care decalarea chiar functioneaza? Care, si de ce tocmai aceea?
3. **B5.** `Linie` , `Dreptunghi` si `Cerc` n-au avut nicio greseala in `translate` , si totusi niciuna nu muta corect. Ce iti spune asta despre unde trebuie sa testezi primul intr-o ierarhie de clase?

Runda 1 — 2026-09-16 — sha 6444afb

Textul integral al rundeii 1 e arhivat la profesor in `ARHIVA-runda1-integral.md` .

-  B1 proiectul nu compileaza (`rol has protected access`) — **inchis** in runda 2
-  B2 un `Admin` salvat dispare la repornire (`loadData` fara `case "ADMIN"`) — **NEATINS**
-  B3 o linie stricata in `users.txt` sterge restul fisierului (`try` peste tot `while` -ul) — **NEATINS**
-  B4 doua cerinte lipsa — `numarStudent()` **scris**, exercitiul 8 (`ClassCastException`) tot lipseste
-  M1 exercitiile 2 si 3 amestecate in aceeasi bucla — NEATINS
-  M2 `nume` duplicat in cele trei subclase — NEATINS
-  M3 `cautaUserDupaId` si cauta si tipareste, rezultatul e aruncat — NEATINS
-  M4 `Scanner` neinchis — **inchis**, e in `try (...)`
- C1–C10 cleanups — partial inchise odata cu rescrierea fisierelor

